YAPAY ZEKÂ

ORTAOKUL

DENEYAP TÜRKİYE

#MİLLİ TEKNOLOJİ HAMLESİ

İSTANBUL DENEYAP TEKNOLOJİ ATÖLYESİ

YAPAY ZEKA DERSİ

8. Hafta: Derin Öğrenme ile İleri Düzey Çözümler

Advanced Solutions with Deep Learning



T3 DENEYAP EĞİTMENİ (Hazırlayan ve Sunan): Muhammed Emin Korkunç

(Bilgisayar Mühendisi)

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

DERS SAATLERİ : 19.30-20.20 = 1.DERS (50dk)
20.20-20.30 = 1.MOLA (10dk)

20.30-21.20 = 2.DERS (50dk)
21.20-21.35 = 2.MOLA (15dk)

21.35-22.30 = 3.DERS (55dk)

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.
Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

1.ALGILA

1.1. Yapay Sinir Ağları Temelinde Derin Öğrenme'ye Geçiş
1.2. Evrişimsel Sinir Ağları Tekniği
1.3. Derin İnanç Ağları Tekniği
1.4. Uzun-Kısa Dönem Hafıza Tekniği

2. TASARLA

3. HAREKETE GEÇ

4. YÜRÜT

5. KARAR VER

5.1. Sonuçların Gözlemlenmesi

6. İLAVE ETKİNLİK

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

8. Hafta: Derin Öğrenme (Deep Learning) ile İleri Düzey Çözümler

- **Büyük Soru: Bir bilgisayar, binlerce resim arasından bir kediyi köpeği nasıl şak diye ayırabilir?**
- **Bu Haftaki Görevimiz: Sadece bir "kod" yazmayacağız; bir bilgisayara "görmeyi" ve "karar vermeyi" öğreteceğiz!**
- **Hedefimiz: Dünyaca ünlü ResNet mimarisini kullanarak kedi ve köpekleri ayırt eden süper bir yapay zekâ modeli kurmak.**

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

3-Bu Haftaki Süper Güçlerimiz (Kazanımlar)

Neler Öğreneceğiz?

Bu ders bittiğinde şunları yapabiliyor olacaksın:

- **Derin Öğrenme Uzmanı: Derin öğrenme modelinin ne olduğunu ve nasıl çalıştığını kavrayacaksın.**
- **Mimarileri Tanı: AlexNet, GoogleNet ve ResNet gibi dev isimlerin dünyasına gireceksin.**
- **Python ile Tasarla: Büyük veri setlerini (binlerce fotoğrafı) yapay zekâya tanıtmayı öğreneceksin.**
- **Geleceği Yorumla: Elde ettiğin sonuçların ne anlama geldiğini bir veri bilimcisi gibi tartışabileceksin.**

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

4- Yol Haritamız: Nasıl İlerleyeceğiz?

5 Adımda Derin Öğrenme Yolculuğu

1. 👀 **Algıla: Derin öğrenmenin gizemli mimarisini keşfedeceğiz.**
2. 🎨 **Tasarla: Kedi ve köpekleri birbirinden ayıracak çözüm planımızı hazırlayacağız.**
3. 🚀 **Harekete Geç: Python ve Github kullanarak ResNet mimarisini hayata geçireceğiz.**
4. 🤖 **Yürüt: Modelimizi çalıştırıp eğitimin nasıl gittiğini canlı canlı izleyeceğiz.**
5. ⚖️ **Karar Ver: Sonuçları değerlendirip "Yapay Zekâ bu işi ne kadar iyi başardı?" diye tartışacağız.**

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

Derse Başlamadan Önce Hatırlayalım

- Derin Öğrenme: Temellerini geçen derslerden biliyoruz, şimdi üzerine kat çıkma vakti!
- Python Bilgisi: Döngüler, fonksiyonlar ve o meşhur kütüphaneler (Numpy, PyTorch vb.) bugün en büyük yardımcımız olacak.
- Malzemeler: Bilgisayarın hazır mı? Kod editörün açık mı? O zaman başlıyoruz!

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

1.1. Derin Öğrenme Nedir?

- **Büyük Resim: Derin öğrenme, aslında Makine Öğrenmesi'nin bir alt dalıdır; o da Yapay Zekâ'nın kalbidir.**
- **İlham Kaynağı: İnsan beynindeki sinir hücrelerinin çalışma şeklini taklit eden "Yapay Sinir Ağları"nı kullanır.**
- **Süper Gücü: Metinler, sesler ve özellikle resimler gibi karmaşık verileri (yapılandırılmamış veri) bir insan gibi analiz edip kararlar alabilir.**



Şekil 8.1. Yapay zekâ ve alt dalları



Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

7-Neden "Derin" Diyoruz?

Katmanların Gücü

- Çok Katmanlı Yapı: Yapay sinir ağlarındaki katman sayısı arttıkça model daha karmaşık şeyleri öğrenmeye başlar. İşte bu yüzden ona "Derin" öğrenme diyoruz.
- Hız ve Doğruluk: Derin öğrenme mimarileri, devasa veri setlerini (Büyük Veri) çok daha hızlı ve doğru analiz etmek için tasarlanmıştır.
- Mimariler: Tıpkı bir binanın projesi gibi, derin öğrenmenin de farklı işler için uzmanlaşmış "mimari" tipleri vardır.

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

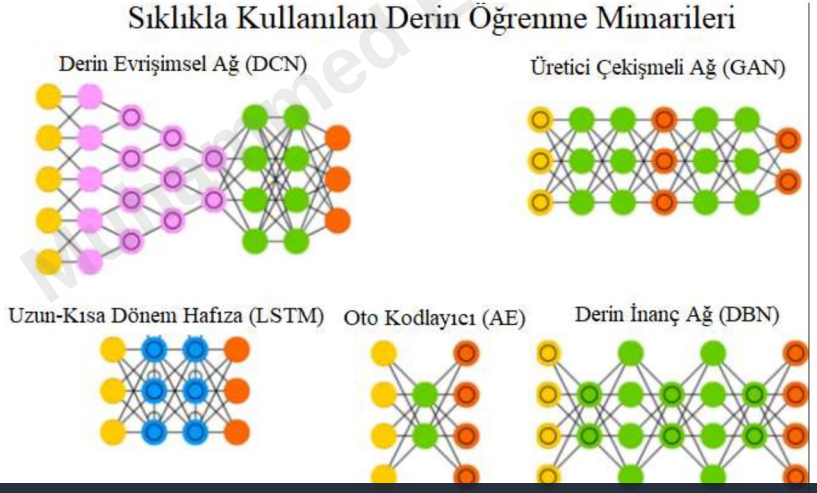
Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

8- Derin Öğrenme Mimarileri Galerisi

Farklı Sorunlar İçin Farklı Çözümler

Bilim insanları farklı görevler için özel mimariler geliştirmiştir:

- **DCN (Derin Evrişimsel Ağlar): Görüntüleri tanımak için uzmanlaşmış "gözler".**
- **LSTM (Uzun-Kısa Dönem Hafıza): Zamanı ve sıralamayı unutmayan "hafıza".**
- **GAN (Üretici Çekişmeli Ağlar): Yeni resimler veya sesler üreten "yaratıcı zihin".**



Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

9-Öğrendiklerimizi Test Edelim! (Soru-Cevap)

Soru 1: Derin öğrenme ile makine öğrenmesi arasındaki fark nedir?

- **Cevap: Derin öğrenme, makine öğrenmesinin bir parçasıdır ama daha fazla katman kullanarak çok daha karmaşık (resim, ses gibi) verileri kendi başına analiz edebilir.**

Soru 2: Neden çok fazla katman kullanıyoruz?

- **Açıklama/Cevap: Tıpkı bir dedektifin ipuçlarını katman katman incelemesi gibi; ilk katmanlar basit şekilleri görürken, daha derin katmanlar kedi veya köpek gibi bütün bir nesneyi tanımaya başlar.**

Soru 3: Büyük veri nedir?

- **Cevap: Milyonlarca kedi fotoğrafı, binlerce saatlik ses kaydı veya devasa kütüphaneler dolusu yazıya "Büyük Veri" diyoruz. Derin öğrenme bu büyük yığının içinden anlamlı bilgiyi bulup çıkarır.**

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

1.2. Evrişimsel Sinir Ağları (CNN)

- **Nedir?: Görüntüleri analiz etmek ve nesneleri (kedi, köpek, araba vb.) birbirinden ayırt etmek için kullanılan süper bir derin öğrenme mimarisidir.**
- **Kullanım Alanları:**
  - **Tıp: Beyin tümörü tespiti.**
  - **Güvenlik: Yüz tanıma sistemleri.**
  - **Otomotiv: Kendi kendine giden (otonom) araçlar.**
  - **Sosyal Medya: Fotoğraflardaki arkadaşlarınızı otomatik etiketleme.**

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

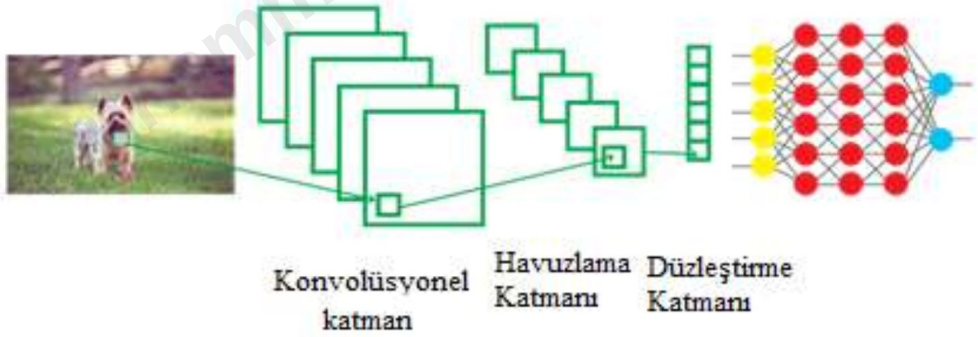
Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

11-CNN'in Katmanları

Bir Görüntü Hangi Aşamalardan Geçer?

Bir fotoğrafın "kedi" olduğunu anlamak için bilgisayar şu adımları izler:

1. **Konvolüsyonel (Evrişim) Katmanı: Fotoğraftaki renkleri, çizgileri ve şekilleri saptayan dedektif katmandır.**
2. **Havuzlama Katmanı: Gereksiz detayları (örneğin arka plandaki ağaçları) silip, sadece önemli bilgileri (köpeğin kendisi gibi) tutarak işlemi hızlandırır.**
3. **Düzleştirme Katmanı: Gördüğü 2 boyutlu resmi, bilgisayarın hesaplayabileceği tek sıra halindeki sayılara dönüştürür.**
4. **Tam Bağımlı Katman: En önemli aşamadır! Toplanan tüm bilgileri birleştirip "Bu kesinlikle bir köpektir!" kararını verir. Tam bağımlı katman, CNN mimarisinin son ve en önemli katmanlardan birisidir. Tam bağımlı katman ile sınıflandırma veya regresyon işlemleri için eğitimler gerçekleştirilir. Örneğin görüntüden çıkartılan özelliklerin kedi köpek vs. gibi hangi hayvana ait olduğunu sınıflandırılmak için kullanılır.**



Şekil 8.3. CNN mimarisi (Web Kaynağı 8.2)

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

Dünyaca Ünlü Yapay Zekâ Modelleri

Tıpkı farklı araba modelleri gibi, derin öğrenmenin de ünlü modelleri vardır:

- 📜 LeNet: 1990'larda çeklerin üzerindeki sayıları okumak için doğdu. CNN'in ilk başarılı örneğidir.
- 🔥 AlexNet: 2012'de derin öğrenmeyi tüm dünyaya tanıtan efsane modeldir.
- 🔍 GoogLeNet: Google tarafından geliştirilen, çok hızlı ve akıllı bir yüz tanıma ustasıdır.
- 🧠 VGGNet: Çok derin katmanlara sahip bir nesne tanıma uzmanıdır. Beyin tümörü tespiti gibi ciddi işlerde kullanılır.

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

Yapay Zekâ Hastalıklara Karşı!

- **COVID-19 Savaşçısı: Derin öğrenme, DNA ve protein yapılarını inceleyerek virüslerle savaşan araştırmacılara büyük destek olmuştur.**

- **Hücre Analizi: İnsan vücudundaki milyarlarca hücrenin hiyerarşisini ve karmaşık hastalık ağlarını çözmek için bu modeller kullanılır.**

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

14-Öğrendiklerimizi Test Edelim! (Soru-Cevap)

Soru 1: Havuzlama katmanı neden fotoğraftaki arka planı (örneğin bahçeyi) yok sayar?

- **Cevap: Çünkü bizim hedefimiz sadece hayvanın ne olduğunu anlamaktır. Arka plan gereksiz bir veri kalabalığıdır ve bilgisayarı yavaşlatır.**

Soru 2: "Düzleştirme" işlemi neden gereklidir?

- **Açıklama/Cevap: Bilgisayarlar resimleri bir tablo (matris) gibi görür. Son aşamada karar verebilmek için bu tabloyu uzun bir sayı listesi haline getirmemiz gerekir.**

Soru 3: İlk başarılı CNN uygulaması (LeNet) ne zaman ve ne amaçla kullanılmıştır?

- **Cevap: 1990'larda, posta kodlarını ve banka çeklerindeki el yazısı rakamları okumak için kullanılmıştır.**

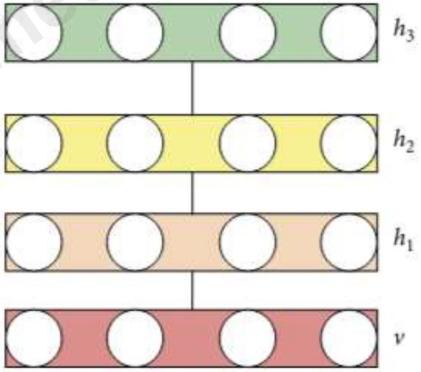
Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

1.3. Derin İnanç Ağları: Katmanların Gücü

- **Nedir?: Birbirine bağlı birden fazla "gizli" katmandan oluşan çok özel bir derin sinir ağı türüdür.**
- **Nasıl Çalışır?: Diğer ağlardan farkı, düğümlerin tek tek değil, katmanların bloklar halinde birbirine bağlanmasıdır.**
- **Görünmez Bilgi: Şekil 8.4'te gördüğümüz yeşil, sarı ve turuncu katmanlar (h1, h2, h3), verinin içindeki gizli özellikleri adım adım ortaya çıkarır.**



Şekil 8.4. DBN model şeması (Dai et al., 2020).

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

LSTM: Asla Unutmayan Bilgisayarlar

- **Neden Var?: Normal yapay sinir ağları bazen çok eski bilgileri "unutabilir" (bu soruna *vanishing gradient* denir).**

- **Çözüm: 1997'de geliştirilen LSTM, bilgiyi uzun süre saklayabilen bir "kanal" açarak öğrenmenin hiç durmamasını sağlar.**

- **Kullanım Alanı: Çeviri programları (cümleyi başından sonuna hatırlar) ve borsa "zamanla değişen" işlerde bir numaradır.**

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

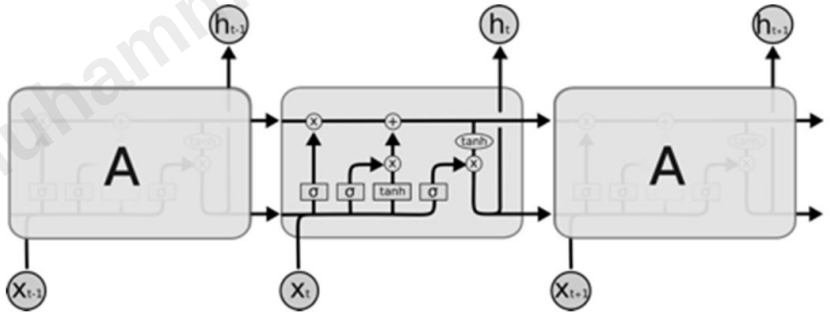
Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

17- Akıllı Kapılar (Hücrenin Karar Anı)

Hücrenin İçinde Neler Oluyor? (Şekil 8.5)

LSTM hücresi, tıpkı bir kütüphaneci gibi çalışır:

1. **Hücre Hafızası: Bilgiyi saklar, yazar veya okur.**
2. **Özel Kapılar: 4 farklı kapıdan oluşur. Bu kapılar şunlara karar verir:**
   - **Neyi Hatırlayalım?: Önemli bilgileri hücreye kaydeder.**
   - **Neyi Unutalım?: Artık gereksiz olan bilgileri siler.**
   - **Dışarıya Ne Verelim?: Hangi bilginin sonuç olarak paylaşılacağına karar verir.**



Şekil 8.5 LSTM Network Yapısı (Web Kaynağı 8.4)

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

18-Öğrendiklerimizi Test Edelim! (Soru-Cevap)

Soru 1: DBN mimarisinde (Şekil 8.4) katmanlar arasında nasıl bir bağ vardır?

- **Cevap: DBN'de sadece yan yana duran düğümler değil, bütün katmanlar birbirine bağlı bir yapı sunar.**

Soru 2: LSTM neden normal bir nöron gibi çalışmaz?

- **Açıklama/Cevap: Çünkü normal nöronlar bilgiyi hemen işleyip geçer. LSTM ise "kapılar" sayesinde hangi bilgiyi saklayacağına ve ne zaman sileceğine karar verebilen bir hafıza sistemine sahiptir.**

Soru 3: LSTM neden çeviri (tercüme) uygulamalarında çok başarılıdır?

- **Cevap: Çünkü bir cümleyi çevirirken cümlenin başındaki özneyi sonuna kadar "hatırlaması" gerekir. LSTM'in uzun dönem hafızası bunu sağlar.**

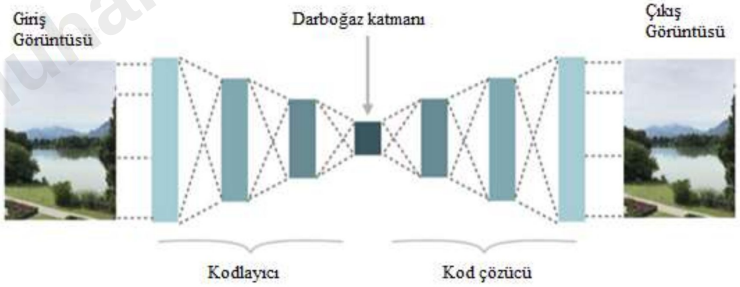
Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

1.5. Oto Kodlayıcı (Auto Encoder) Tekniği

- **Nedir?: Veriyi otomatik olarak öğrenen ve onu çok daha küçük bir "gizli koda" dönüştüren popüler bir derin öğrenme modelidir.**
- **İki Ana Kahraman:**
  - **Kodlayıcı (Encoder): Giriş verisini (örneğin bir fotoğrafı) analiz eder ve onu küçük, yoğun bir koda dönüştürür.**
  - **Kod Çözücü (Decoder): Bu küçük koddan yola çıkarak orijinal veriyi tekrar oluşturmaya çalışır.**
- **Neden Kullanırız?: Kişisel görüntülerimizi güvenli bir şekilde şifreleyerek bir yerden başka bir yere taşımak için mükemmeldir.**



Şekil 2.6. Oto kodlayıcı tekniği şematiği (Web Kaynağı 2.5)

Muhammed Emin Korkunç

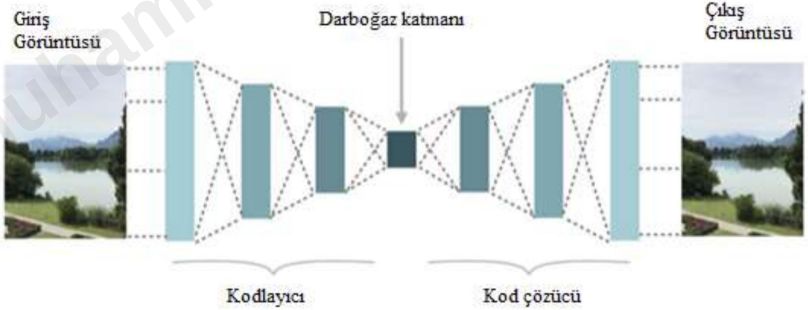
Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

20-Şekil 8.6: Verinin Darboğazdan Geçişi

Bir Görüntü Nasıl Şifrelenir?

Görselimizdeki süreci adım adım inceleyelim:

1. **Giriş Görüntüsü: Orijinal manzara fotoğrafımız sisteme girer.**
2. **Kodlayıcı: Fotoğrafı gitgide küçülterek en önemli özelliklerini ayıklar.**
3. **Darboğaz Katmanı: Burası en kritik yer! Fotoğraf artık minicik bir "sayı dizisi" (kod) halindedir.**
4. **Kod Çözücü: Bu minik kodu kullanarak fotoğrafı tekrar eski haline getirir.**



Şekil 8.6. Oto kodlayıcı tekniği şematiği (Web Kaynağı 8.5)

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

Cebinizdeki Yapay Zekâ (Web Kaynağı 8.6)

Günlük hayatta kullandığınız birçok uygulamanın arkasında aslında Derin Öğrenme var:

- 🤳 FaceApp: Fotoğraflarınıza gerçekçi yaşlandırma veya gülümseme filtreleri ekler.
- 🗣️ Siri: Sizinle konuşan ve sizi anlayan akıllı asistan.
- 🎭 Snapchat & Instagram: Yüzünüze tam oturan o eğlenceli maskeler.
- 🔍 Google Görseller: Bir fotoğraf yüklediğinizde benzerlerini bulan sistem.

Biliyor musunuz?: FaceApp uygulaması yayınlandığı ilk 2 haftada tam 1 milyon iPhone kullanıcısı tarafından indirilmiştir!

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

22-Öğrendiklerimizi Test Edelim! (Soru-Cevap)

Soru 1: Oto kodlayıcılarda "Darboğaz" (Bottleneck) katmanı neden bu kadar küçüktür?

- **Cevap: Bilgisayarın veriyi ezberlemesini değil, en önemli ve en öz bilgiyi öğrenmesini istediğimiz için onu bu dar kapıdan geçmeye zorlarız.**

Soru 2: Fotoğraflarımızı bir yerden başka bir yere taşırken neden Oto Kodlayıcı kullanmak güvenlidir?

- **Açıklama/Cevap: Çünkü taşıma sırasında veri orijinal fotoğraf değil, sadece anlamsız görünen bir "kod" halindedir. Kod çözücüye sahip olmayan biri fotoğrafı göremez.**

Soru 3: Derin öğrenme sadece eğlence için mi kullanılır?

- **Cevap: Hayır! "Biliyor Musunuz?" kısmında gördüğümüz gibi, COVID-19 araştırmalarında DNA ve hücre yapılarını incelemek gibi hayati görevlerde de başrol oynar.**

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

akın Özellikleri Ayırt Etmek

- Büyük Mücadele: Kediler ve köpekler birbirine çok benzer (4 ayak, kuyruk, kulak sayısı...). Normal makine öğrenmesi bu ikisini karıştırabilir.
- Derin Çözüm: Derin öğrenme sayesinde en ince detayları (göz yapısı, tüy dokusu vb.) öğrenerek bu sorunu aşıyoruz.
- Veri Gücümüz: Tam 25.000 adet kedi ve köpek görüntüsü kullanacağız!
- Modelimiz: Dünyaca ünlü bir CNN türü olan ResNet152_V2.



Şekil 9.7. Veri setindeki örnek görüntü

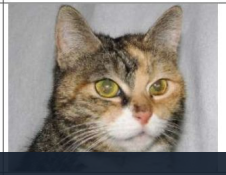
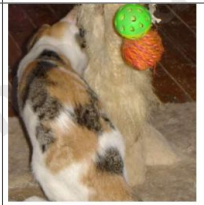
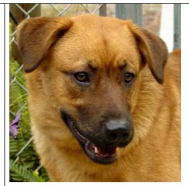
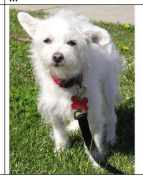
Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

Çizelge 8.1: Bilgisayar Ne Alır, Ne Verir?

- Giriş (Input): Kedi veya köpek fotoğrafı.
- Çıkış (Output): Bilgisayar bize bir numara söyler:
  - 0: Kedi 🐱
  - 1: Köpek 🐶

Veri Sayısı	GİRİŞ PARAMETRESİ	ÇIKIŞ PARAMETRESİ
	Kedi köpek görüntüsü	Sınıf
1		0 (Kedi)
1006		0(Kedi)
...	...	...
12500		0 (Kedi)
12501		1 (Köpek)
...	...	...
13025		1 (Köpek)
...	...	...
25000		1 (Köpek)

Muhammed Emin Korkunç

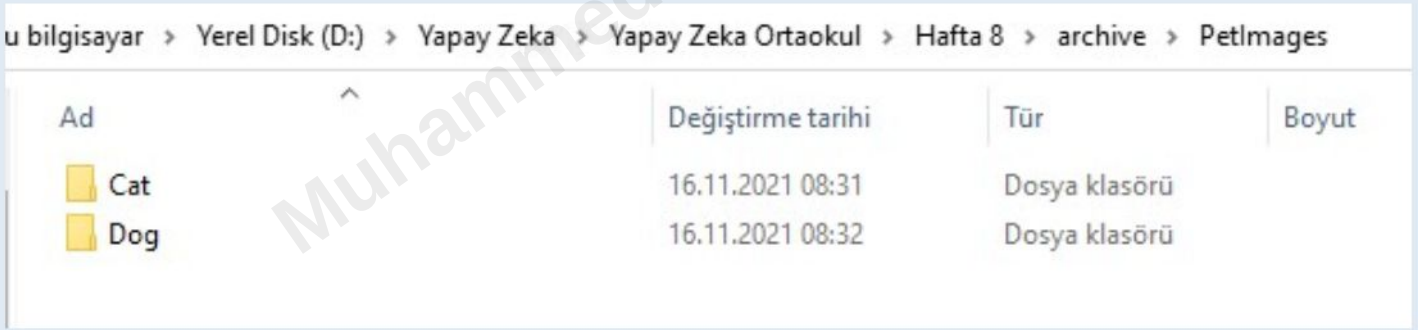
Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

25- Giriş ve Çıkış Parametrelerimiz

Veri Seti İndirme Linki = https://www.kaggle.com/shaunthesheep/microsoft-catsvsdogs-dataset

Eğitmene Not

Eğitmen, Şekil 8.8'de gösterildiği gibi öğrencilere veri setini anlatırken linkten indirilen verilerin Testing ve Training olarak ayrı bir şekilde indirildiğini anlatır.



Şekil 8.8. Veri seti klasörleri

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

26- Harekete Geç-1

Ders_Kodları= https://www.kaggle.com/code/muhammedkorkun/t3-deneyap/edit

```
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import tensorflow as tf
from tqdm import tqdm
import os
from sklearn.metrics import classification_report, confusion_matrix
from PIL import Image
from tensorflow.keras.applications import ResNet152V2
from sklearn.model_selection import train_test_split
from warnings import filterwarnings

filterwarnings('ignore')
print("Kütüphaneler yüklendi.")
```

```
Kütüphaneler yüklendi.
```

Muhammed Emin Korkunç

matplotlib & seaborn: Eğitim sonuçlarını ve başarı tablolarını görselleştiren grafik araçlarıdır.
numpy: Resimleri sayısal matrislere dönüştürüp matematiksel hesaplama yapmamızı sağlar.
tensorflow & ResNet152V2: Yapay zekanın beynidir; hazır devasa bir modeli kullanarak öğrenmeyi sağlar.
tqdm: Binlerce resim yüklenirken işlemin ne kadar bittiğini gösteren ilerleme çubuğudur.
os: Klasör yollarına erişmek ve dosya listesini almak için kullanılır.
sklearn: Veriyi eğitim/test olarak böler ve başarı raporu (yüzde kaç doğru bildik) sunar.
PIL (Image): Fotoğrafları açıp boyutlarını (örneğin 64x64) ayarlayan resim editörüdür.
filterwarnings: Kodun çalışmasını etkilemeyen gereksiz uyarı mesajlarını gizler.

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.
Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

27- Harekete Geç-2

```
base_path = '/kaggle/input/microsoft-catsvsdogs-dataset/PetImages'
labels = ['Cat', 'Dog']
image_size = 64
X_data = []
y_data = []

# RAM dostu olması için her sınıftan 8000 resim alalım (toplam 16.000)
limit_per_class = 8000

for i in labels:
    folderPath = os.path.join(base_path, i)
    files = os.listdir(folderPath)[:limit_per_class]
    for j in tqdm(files, desc=f"Yükleniyor: {i}"):
        yol = os.path.join(folderPath, j)
        try:
            img = Image.open(yol).convert('RGB')
            img = img.resize((image_size, image_size), Image.LANCZOS)
            # Normalizasyon: 255'e bölerek pikselleri 0-1 arasına çekiyoruz
            X_data.append(np.array(img) / 255.0)
            y_data.append(i)
        except:
            continue

X_data = np.array(X_data)
y_data = np.array(y_data)
print(f"\nVeri hazırlığı tamamlandı. Toplam resim: {len(X_data)}")
```

```
Yükleniyor: Cat: 100%|██████████| 8000/8000 [00:28<00:00, 282.48it/s]
Yükleniyor: Dog: 100%|██████████| 8000/8000 [00:28<00:00, 278.63it/s]

Veri hazırlığı tamamlandı. Toplam resim: 15999
```

- **`base_path` & `labels`**: Resimlerin konumunu ve kategorilerini (Kedi/Köpek) tanımlar.
- **`image_size = 64`**: Tüm resimlerin ayarlanacağı standart genişlik ve yükseklik değeridir.
- **`X_data` & `y_data`**: Resim piksellerini ve etiketlerini saklamak için boş listeler oluşturur.
- **`limit_per_class = 8000`**: RAM'in dolmasını önlemek için her sınıftan alınacak dosya sayısını sınırlar.
- **`for i in labels`**: Kedi ve Köpek klasörleri üzerinde sırayla döngü başlatır.
- **`os.path.join` & `os.listdir`**: Klasör yolunu birleştirir ve içindeki dosyaların listesini alır.
- **`tqdm`**: Yükleme işleminin ilerleyişini gösteren bir çubuk ekler.
- **`Image.open().convert('RGB')`**: Resmi açar ve tüm görselleri standart 3 kanallı renkli formata getirir.
- **`resize`**: Resmi modelin beklediği **64x64** boyutuna ölçekler.
- **`np.array(img) / 255.0`**: Pikselleri 0-1 arasına çeker (normalizasyon); bu, modelin çok daha hızlı öğrenmesini sağlar.
- **`X_data.append` & `y_data.append`**: İşlenen resmi ve ismini listelere kaydeder.
- **`try-except`**: Bozuk resim dosyalarını atlayarak kodun çökmesini engeller.
- **`np.array()`**: Listeleri yüksek performanslı matematiksel dizilere dönüştürür.

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

28- Harekete Geç-3

```
X_train, X_test, y_train, y_test = train_test_split(X_data, y_data, test_size=0.1, random_state=101)

def kodlama(y_t):
    y_new = []
    for i in y_t:
        y_new.append(labels.index(i))
    return tf.keras.utils.to_categorical(y_new, num_classes=2)

y_train = kodlama(y_train)
y_test = kodlama(y_test)

print("Eğitim ve test setleri hazır.")
```

```
Eğitim ve test setleri hazır.
```

train_test_split: Verinin %90'ını eğitim, %10'unu test (sınav) yapmak üzere rastgele ayırır.
random_state=101: Her çalıştırmada aynı rastgele ayrımın yapılmasını sağlayarak sonuçları tutarlı kılar.
def kodlama(y_t): "Cat" veya "Dog" kelimelerini bilgisayarın anlayacağı sayılara çeviren bir fonksiyon tanımlar.
labels.index(i): "Cat" yazısını `0`, "Dog" yazısını `1` değerine dönüştürür.
to_categorical: Sayıları **One-Hot Encoding** formatına (Kedi için `[1, 0]`, Köpek için `[0, 1]`) çevirir.
y_train = kodlama(...): Bu işlemi tüm eğitim ve test etiketlerine uygulayarak veriyi modele hazır hale getirir.

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.
Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

29- Harekete Geç-4

```
# Hazır modeli indiriyoruz
resnet = ResNet152V2(weights='imagenet', include_top=False, input_shape=(image_size, ima
resnet.trainable = False  # Temel ağırlıkları dondur

model_out = resnet.output
model_out = tf.keras.layers.GlobalAveragePooling2D()(model_out)
model_out = tf.keras.layers.BatchNormalization()(model_out)
model_out = tf.keras.layers.Dense(256, activation='relu')(model_out)
model_out = tf.keras.layers.Dropout(rate=0.5)(model_out)
predictions = tf.keras.layers.Dense(2, activation='softmax')(model_out)

model = tf.keras.models.Model(inputs=resnet.input, outputs=predictions)

model.compile(loss='categorical_crossentropy', optimizer='Adam', metrics=['accuracy'])
print("Model dondurulmuş katmanlarla hazır.")
```

- **`ResNet152V2(...)`**: Milyonlarca resimle önceden eğitilmiş devasa bir modeli (ResNet152) temel yapı olarak indirir.
- **`resnet.trainable = False`**: İndirilen modelin mevcut bilgilerini bozmamak için ana ağırlıklarını dondurur (değişmez kılar).
- **`GlobalAveragePooling2D`**: Karmaşık veriyi sadeleştirerek modelin daha genel özellikleri öğrenmesini sağlar.
- **`BatchNormalization`**: Verileri normalize ederek eğitimin daha hızlı ve kararlı ilerlemesine yardımcı olur.
- **`Dense(256, ...)`**: Öğrenilen özelliklerin analizi için 256 nöronluk yeni bir ara karar katmanı ekler.
- **`Dropout(0.5)`**: Nöronların yarısını rastgele kapatarak modelin ezber yapmasını (overfitting) önler.
- **`Dense(2, activation='softmax')`**: En son katmanda resmin % kaç olasılıkla kedi veya köpek olduğuna karar verir.
- **`tf.keras.models.Model`**: ResNet girişi ile bizim eklediğimiz çıkış katmanlarını tek bir bütün haline getirir.
- **`model.compile`**: Hata hesaplama yöntemini (`loss`), optimize ediciyi (`Adam`) ve başarı ölçütünü (`accuracy`) belirleyerek modeli eğitime hazırlar.

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylaşılması ve yazılı izinsiz kullanımı KESİNLİKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

30- Yürüt

```python
# Eğitimi başlat
history = model.fit(X_train, y_train, validation_split=0.1, epochs=5, verbose=1, batch_size=64)

# Tahminler ve Rapor
pred = model.predict(X_test)
pred_classes = np.argmax(pred, axis=1)
y_test_classes = np.argmax(y_test, axis=1)

print("\n--- Sınıflandırma Raporu ---")
print(classification_report(y_test_classes, pred_classes, target_names=labels))

# Karmaşıklık Matrisi
plt.figure(figsize=(8, 6))
sns.heatmap(confusion_matrix(y_test_classes, pred_classes), annot=True, fmt='d', cmap='Blues', xticklabels=labels, yticklabels=labels)
plt.title('Karmaşıklık Matrisi')
plt.xlabel('Tahmin Edilen')
plt.ylabel('Gerçek')
plt.show()
```

- **`model.fit`**: Eğitimi başlatır; verileri modele göstererek öğrenmesini sağlar ve her adımda başarısını takip eder.
- **`model.predict`**: Eğitilen modeli "sınava sokar"; test setindeki resimlerin kedi mi köpek mi olduğuna dair tahminler üretir.
- **`np.argmax`**: Modelin ürettiği olasılık puanları arasından en yüksek olanı seçerek net bir sınıfa (0 veya 1) karar verir.
- **`classification_report`**: Modelin başarı istatistiklerini (doğruluk oranı, hata payı vb.) içeren detaylı bir özet sunar.
- **`plt.figure`** & **`sns.heatmap`**: Hangi resimlerin doğru bilindiğini, hangilerinin karıştırıldığını gösteren renkli ve görsel bir tablo oluşturur.
- **`plt.title`** & **`plt.show()`**: Hazırlanan grafiğe başlık ekler ve sonucu ekranda görüntüler.

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

31- Karar Ver

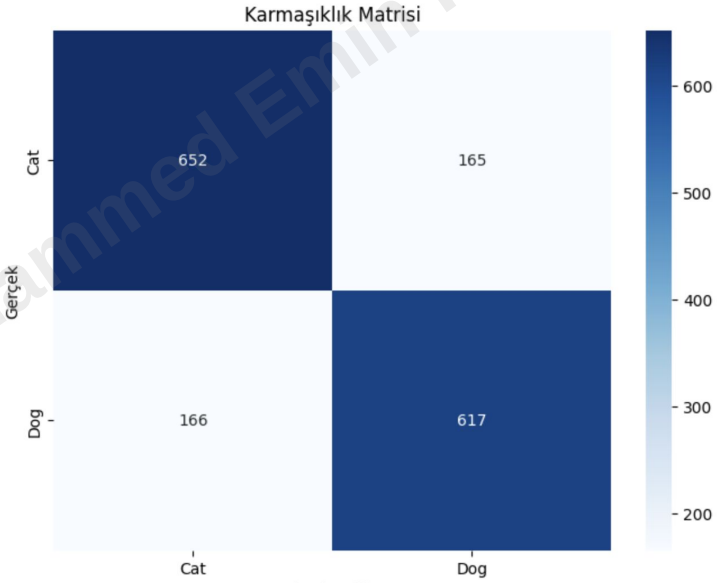
```
Epoch 1/5
203/203 ━━━━━━━━━━━━━━━━━━━━ 8s 42ms/step - accuracy: 0.8223 - loss: 0.3858 - val_accuracy: 0.8153 - val_loss: 0.3873
Epoch 2/5
203/203 ━━━━━━━━━━━━━━━━━━━━ 8s 41ms/step - accuracy: 0.8495 - loss: 0.3379 - val_accuracy: 0.8194 - val_loss: 0.3897
Epoch 3/5
203/203 ━━━━━━━━━━━━━━━━━━━━ 8s 41ms/step - accuracy: 0.8593 - loss: 0.3081 - val_accuracy: 0.8319 - val_loss: 0.3933
Epoch 4/5
203/203 ━━━━━━━━━━━━━━━━━━━━ 8s 41ms/step - accuracy: 0.8797 - loss: 0.2712 - val_accuracy: 0.8340 - val_loss: 0.3902
Epoch 5/5
203/203 ━━━━━━━━━━━━━━━━━━━━ 8s 41ms/step - accuracy: 0.9041 - loss: 0.2350 - val_accuracy: 0.8188 - val_loss: 0.4231
50/50 ━━━━━━━━━━━━━━━━━━━━ 1s 28ms/step
```

```
--- Sınıflandırma Raporu ---
              precision    recall  f1-score   support

         Cat       0.80      0.80      0.80       817
         Dog       0.79      0.79      0.79       783

    accuracy                           0.79      1600
   macro avg       0.79      0.79      0.79      1600
weighted avg       0.79      0.79      0.79      1600
```









Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

32- Karar Ver

```python
import requests
from io import BytesIO

def disaridan_tahmin_et(url):
    try:
        # Resmi internetten indir
        response = requests.get(url)
        img = Image.open(BytesIO(response.content)).convert('RGB')

        # Göstermek için orijinal resmi sakla
        plt.imshow(img)
        plt.axis('off')

        # Modeli eğittiğimiz formata getir (64x64 ve normalize)
        img_resized = img.resize((image_size, image_size), Image.LANCZOS)
        img_array = np.array(img_resized) / 255.0
        img_array = np.expand_dims(img_array, axis=0) # Batch boyutu ekle

        # Tahmin yap
        tahmin = model.predict(img_array)
        sinif_index = np.argmax(tahmin)
        guven_skoru = tahmin[0][sinif_index] * 100

        plt.title(f"Tahmin: {labels[sinif_index]} (%{guven_skoru:.2f} güven)")
        plt.show()
    except Exception as e:
        print("Resim yüklenirken hata oluştu:", e)
```

```python
# TEST ET: Buraya istediğin bir resim linkini yapıştırabilirsin
test_url = "https://www.markamama.com.tr/Data/EditorFiles/blog/sokak-kedisi-1.jpg"
disaridan_tahmin_et(test_url)
```

Muhammed Emin Korkunç

`requests.get(url)`: Verilen internet adresindeki resim dosyasını indirir.
`Image.open().convert('RGB')`: İndirilen veriyi bir resim dosyası olarak açar ve standart renkli (RGB) formata çevirir.
`plt.imshow()` & `plt.axis('off')`: Resmi ekranda gösterir ve kenarlardaki eksen çizgilerini gizler.
`img.resize()`: Resmi, modelin eğitimde öğrendiği **64x64** boyutuna getirir.
`np.array() / 255.0`: Resmi sayısal bir matrise çevirir ve pikselleri **0-1** arasına normalize eder.
`np.expand_dims(..., axis=0)`: Tek bir resmi, modelin beklediği "grup" (batch) formatına uygun hale getirmek için bir boyut ekler.
`model.predict()`: Hazırlanan resmi yapay zeka modeline göndererek tahmin sonuçlarını alır.
`np.argmax(tahmin)`: En yüksek olasılığa sahip olan sınıfın (Kedi veya Köpek) dizin numarasını seçer.
`guven_skoru`: Modelin yaptığı tahminden ne kadar emin olduğunu yüzde (%) cinsinden hesaplar.
`plt.title()` & `plt.show()`: Tahmin sonucunu ve güven oranını resmin üzerine başlık olarak yazar ve ekrana basar.
`try-except`: Bağlantı kopması veya hatalı URL gibi

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

33-Beyin Tümörü Sınıflandırması (MRI)

Derin Öğrenme ile Medikal Teşhis: Beyin Tümörü Sınıflandırması

- **Problem Tanımı: Beyin tümörü, manuel incelemede karmaşıklık nedeniyle hata payı yüksek olan ölümcül bir hastalıktır.**
- **Veri Seti (Kaggle): 3264 adet MRI görüntüsü içeren açık erişimli veri seti.**
- **Sınıflandırma Etiketleri:**
  1. **`glioma_tumor` (Glioma)**
  2. **`no_tumor` (Tümör yok)**
  3. **`meningioma_tumor` (Meningiom)**
  4. **`pituitary_tumor` (Hipofiz).**
- **Yöntem (ResNet152V2): Kedi-köpek uygulamasında olduğu gibi, 152 katmanlı derin Konvolüsyonel Sinir Ağı (CNN) mimarisi kullanılmıştır.**
- **Amaç: Radyologlara yardımcı olacak, insan kaynaklı hataları minimize eden destekleyici bir teşhis sistemi geliştirmek.**

Veri setini indirme linki =
https://www.kaggle.com/sartajbhuvaji/brain-tumor-classification-mri

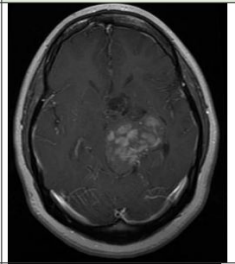
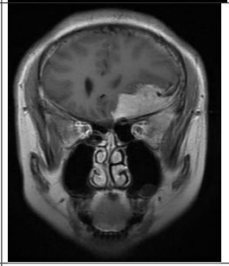
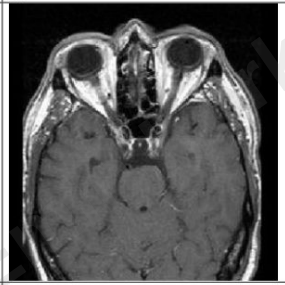
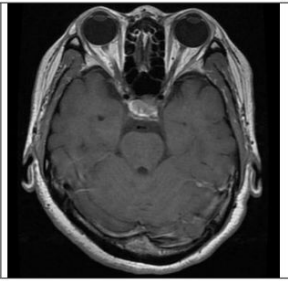
Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

34- Giriş ve Çıkış Parametrelerimiz

Çizelge 8.2. ResNet152_V2 derin öğrenme tekniği için giriş ve çıkış parametreleri

Veri Sayısı	GİRİŞ PARAMETRESİ	ÇIKIŞ PARAMETRESİ
	MRI Görüntüsü	Tümör Sınıfı
1		Glioma_tumor
150		Meningioma_tumor
1440		No_tumor
3264		Pituitary_tumor

Muhammed Emin Korkunç

kaggle kodlar =
https://www.kaggle.com/code/muhammedkorkun/notebook3f45bf1eee/edit

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.
Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

35- Harekete Geç-1

```
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import tensorflow as tf
from tqdm import tqdm
import os
from PIL import Image
from sklearn.utils import shuffle
from sklearn.model_selection import train_test_split
from tensorflow.keras.applications import ResNet152V2
from tensorflow.keras.callbacks import ReduceLROnPlateau, ModelCheckpoint
from sklearn.metrics import classification_report, confusion_matrix
from warnings import filterwarnings

filterwarnings('ignore')
print("Kütüphaneler yüklendi ve GPU kontrol ediliyor...")
```

```
Kütüphaneler yüklendi ve GPU kontrol ediliyor...
```

Bu blok, projenin temel araçlarını hazırladığımız "mutfak" kısmıdır. İşte her satırın kısa ve net görevi:

- `import matplotlib.pyplot as plt`: Eğitim grafiklerini çizmek ve resimleri ekranda göstermek için kullanılır.
- `import numpy as np`: Resimleri sayısal matrislere dönüştürerek matematiksel hesaplamalar yapmamızı sağlar.
- `import seaborn as sns`: Hangi resimlerin doğru veya yanlış bilindiğini gösteren tabloları (Karmaşıklık Matrisi) daha şık görselleştirir.
- `import tensorflow as tf`: Yapay zekanın beynidir; modelin kurulmasını, eğitilmesini ve çalışmasını yönetir.
- `from tqdm import tqdm`: Binlerce resim yüklenirken işlemin yüzde kaçta olduğunu gösteren bir ilerleme çubuğu ekler.
- `import os`: Bilgisayar veya Kaggle üzerindeki dosya ve klasör yollarına erişmek için kullanılır.
- `from sklearn.metrics ...`: Modelin yüzde kaç başarıyla çalıştığını gösteren istatistiksel raporlar üretir.
- `from PIL import Image`: Fotoğrafları açmak ve boyutlarını (örneğin 128x128) değiştirmek için kullanılan resim editörüdür.
- `from tensorflow.keras.applications import ResNet152V2`: Dünyaca ünlü, önceden milyonlarca resimle eğitilmiş hazır yapay zeka mimarisini projeye dahil eder.
- `from sklearn.model_selection import train_test_split`: Veri setini "ders çalışma" (eğitim) ve "sınav yapma" (test) olarak ikiye ayırır.
- `filterwarnings('ignore')`: Kodun çalışmasını etkilemeyen, ekranda kalabalık yapan önemsiz uyarıları gizler.

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

36- Harekete Geç-2

```python
labels = ['glioma_tumor', 'no_tumor', 'meningioma_tumor', 'pituitary_tumor']
image_size = 128
X_data = []
y_data = []

# Kaggle üzerindeki veri yolu
base_path = '/kaggle/input/brain-tumor-classification-mri'

for folder in ['Training', 'Testing']:
    for i in labels:
        folderPath = os.path.join(base_path, folder, i)
        if not os.path.exists(folderPath): continue

        for j in tqdm(os.listdir(folderPath), desc=f"{folder}-{i}"):
            yol = os.path.join(folderPath, j)
            try:
                img = Image.open(yol).convert('RGB')
                img = img.resize((image_size, image_size))
                X_data.append(np.array(img))
                y_data.append(i)
            except:
                continue

X_data = np.array(X_data)
y_data = np.array(y_data)

# Veriyi karıştırarak modelin ezberlemesini önlüyoruz
X_data, y_data = shuffle(X_data, y_data, random_state=101)
print(f"Toplam yüklenen MRI sayısı: {len(X_data)}")
```

```
Training-glioma_tumor: 100%|██████████| 826/826 [00:03<00:00, 224.50it/s]
Training-no_tumor: 100%|██████████| 395/395 [00:01<00:00, 279.73it/s]
Training-meningioma_tumor: 100%|██████████| 822/822 [00:03<00:00, 229.27it/s]
Training-pituitary_tumor: 100%|██████████| 827/827 [00:03<00:00, 215.28it/s]
Testing-glioma_tumor: 100%|██████████| 100/100 [00:00<00:00, 238.54it/s]
Testing-no_tumor: 100%|██████████| 105/105 [00:00<00:00, 457.26it/s]
Testing-meningioma_tumor: 100%|██████████| 115/115 [00:00<00:00, 319.89it/s]
Testing-pituitary_tumor: 100%|██████████| 74/74 [00:00<00:00, 121.41it/s]
Toplam yüklenen MRI sayısı: 3264
```

`labels = [...]`: Teşhis edilecek 4 farklı beyin tümörü kategorisini (Glioma, No Tumor, Meningioma, Pituitary) tanımlar.
`image_size = 128`: Tüm MRI görüntülerinin standart hale getirileceği boyut değeridir.
`X_data = []` & `y_data = []`: Resim piksellerini (X) ve isimlerini (y) saklamak için boş listeler oluşturur.
`base_path = '...'`: Kaggle sistemindeki MRI veri setinin bulunduğu ana klasör yoludur.
`for folder in [...]`: Hem "Training" hem de "Testing" klasörlerini taramak için ana döngüyü başlatır.
`for i in labels`: Kategori klasörleri (tümör tipleri) üzerinde sırayla dolaşır.
`os.path.join(...)`: Bilgisayara dosyanın tam adresini (örn: .../Training/no_tumor) tarif eder.
`os.path.exists(...)`: Aranan klasörün gerçekten orada olup olmadığını kontrol eder.
`tqdm(...)`: MRI'lar yüklenirken ekranın altında işlemin ilerleyişini gösteren bir çubuk çıkartır.
`Image.open().convert('RGB')`: MRI dosyasını açar ve resmi standart 3 kanallı renkli formata çevirir.
`img.resize(...)`: Farklı boyutlardaki MRI'ları modelin beklediği **128x128** boyutuna getirir.
`X_data.append(...)`: İşlenen resim piksellerini sayısal matris olarak listeye ekler.
`y_data.append(i)`: Resmin hangi tümör tipine ait olduğunu listeye kaydeder.
`try-except`: Bozuk veya okunamaz durumdaki MRI dosyalarını atlayarak kodun çökmesini engeller.
`np.array(...)`: Listeleri yüksek performanslı matematiksel dizilere dönüştürür.
`shuffle(...)`: Resimlerin sırasını karıştırarak modelin sınıfları sırayla ezberlemesini önler.
`print(...)`: Toplam kaç adet MRI görüntüsünün başarıyla yüklendiğini ekrana yazar.

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.
Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

37- Harekete Geç-3

```
X_train, X_test, y_train, y_test = train_test_split(X_data, y_data, test_size=0.1, random_state=101)

def kodlama(y_list):
    y_new = [labels.index(i) for i in y_list]
    return tf.keras.utils.to_categorical(y_new)

y_train = kodlama(y_train)
y_test = kodlama(y_test)

print("Veri setleri eğitime hazır hale getirildi.")
```

```
Veri setleri eğitime hazır hale getirildi.
```

train_test_split: Tüm veriyi %90 eğitim (öğrenme) ve %10 test (sınav) olarak ikiye ayırır.
random_state=101: Her çalıştırmada aynı rastgele ayrımı yaparak sonuçların tutarlı olmasını sağlar.
def kodlama(y_list): Metin halindeki tümör isimlerini sayısal verilere çeviren bir fonksiyon tanımlar.
labels.index(i): Liste içindeki isimleri (glioma, no_tumor vb.) $0, 1, 2, 3$ gibi sayısal dizinlere çevirir.
to_categorical: Bu sayıları modelin anlayacağı **One-Hot Encoding** formatına (örneğin 0 yerine [1, 0, 0, 0]) dönüştürür.
y_train = kodlama(...): Hazırlanan bu sayısal dönüşümü eğitim etiketlerine uygular.
y_test = kodlama(...): Aynı dönüşümü test (sınav) etiketlerine de uygulayarak işlemi tamamlar.
print(...): Hazırlık sürecinin bittiğini ve eğitimin başlayabileceğini bildiren onay mesajını yazar.

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

38- Harekete Geç-4

```python
# Önceden eğitilmiş ResNet mimarisini temel alıyoruz
resnet = ResNet152V2(weights='imagenet', include_top=False, input_shape=(image_size, image_size, 3))

model_out = resnet.output
model_out = tf.keras.layers.GlobalAveragePooling2D()(model_out)
model_out = tf.keras.layers.Dropout(rate=0.5)(model_out)
# 4 farklı tümör tipine karar veren son katman
predictions = tf.keras.layers.Dense(4, activation='softmax')(model_out)

model = tf.keras.models.Model(inputs=resnet.input, outputs=predictions)

model.compile(loss='categorical_crossentropy', optimizer='Adam', metrics=['accuracy'])
print("Model yapısı kuruldu.")
```

```
Model yapısı kuruldu.
```

`ResNet152V2(...)`: Milyonlarca resimle önceden eğitilmiş (weights='imagenet') devasa bir modeli temel yapı olarak indirir.
`model_out = resnet.output`: ResNet'in özellik çıkaran ana gövdesini alır ve üzerine yeni katmanlar eklemek için çıkış kapısını açar.
`GlobalAveragePooling2D()`: Karmaşık görsel verileri sadeleştirerek modelin tümörün genel özelliklerine odaklanmasını sağlar.
`Dropout(rate=0.5)`: Eğitim sırasında nöronların yarısını rastgele kapatarak modelin ezber yapmasını (overfitting) önler.
`Dense(4, activation='softmax')`: 4 farklı tümör tipi için olasılık üreten son karar katmanını ekler.
`tf.keras.models.Model`: Hazır ResNet giriş katmanı ile bizim eklediğimiz yeni çıkış katmanlarını birleştirerek tam bir model oluşturur.
`model.compile(...)`: Hata hesaplama yöntemini ve başarı ölçütünü belirleyerek modeli eğitime hazır hale getirir.
`print(...)`: Yapay zeka mimarisinin başarıyla kurulduğunu ekrana yazar.

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

39- Yürüt

```python
# Başarıyı artırmak için özel ayarlar
checkpoint = ModelCheckpoint("brain_tumor_model.h5", monitor="val_accuracy", save_best_only=True, verbose=1)
reduce_lr = ReduceLROnPlateau(monitor='val_accuracy', factor=0.4, patience=2, min_delta=0.001, verbose=1)

history = model.fit(X_train, y_train, validation_split=0.1, epochs=5, batch_size=64,
                    callbacks=[checkpoint, reduce_lr])
```

```
Epoch 1/5
42/42 ━━━━━━━━━━━━━━━━━━━━ 0s 1s/step - accuracy: 0.6361 - loss: 0.9556
Epoch 1: val_accuracy improved from -inf to 0.27891, saving model to brain_tumor_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered lega
cy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
42/42 ━━━━━━━━━━━━━━━━━━━━ 220s 2s/step - accuracy: 0.6387 - loss: 0.9498 - val_accuracy: 0.2789 - val_loss: 766.1816 - learning_rate: 0.0010
Epoch 2/5
42/42 ━━━━━━━━━━━━━━━━━━━━ 0s 270ms/step - accuracy: 0.8662 - loss: 0.3914
Epoch 2: val_accuracy did not improve from 0.27891
42/42 ━━━━━━━━━━━━━━━━━━━━ 12s 280ms/step - accuracy: 0.8665 - loss: 0.3908 - val_accuracy: 0.2789 - val_loss: 33.8342 - learning_rate: 0.0010
Epoch 3/5
42/42 ━━━━━━━━━━━━━━━━━━━━ 0s 270ms/step - accuracy: 0.9111 - loss: 0.2641
Epoch 3: val_accuracy improved from 0.27891 to 0.50680, saving model to brain_tumor_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered lega
cy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
42/42 ━━━━━━━━━━━━━━━━━━━━ 15s 359ms/step - accuracy: 0.9112 - loss: 0.2637 - val_accuracy: 0.5068 - val_loss: 3.0317 - learning_rate: 0.0010
Epoch 4/5
42/42 ━━━━━━━━━━━━━━━━━━━━ 0s 269ms/step - accuracy: 0.9611 - loss: 0.1256
Epoch 4: val_accuracy improved from 0.50680 to 0.52381, saving model to brain_tumor_model.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered lega
cy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
42/42 ━━━━━━━━━━━━━━━━━━━━ 15s 353ms/step - accuracy: 0.9608 - loss: 0.1264 - val_accuracy: 0.5238 - val_loss: 4.2263 - learning_rate: 0.0010
Epoch 5/5
42/42 ━━━━━━━━━━━━━━━━━━━━ 0s 270ms/step - accuracy: 0.9552 - loss: 0.1263
Epoch 5: val_accuracy did not improve from 0.52381
42/42 ━━━━━━━━━━━━━━━━━━━━ 12s 280ms/step - accuracy: 0.9550 - loss: 0.1268 - val_accuracy: 0.3844 - val_loss: 3.6802 - learning_rate: 0.0010
```

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

40- Karar Ver

```python
pred = model.predict(X_test)
pred_classes = np.argmax(pred, axis=1)
y_true = np.argmax(y_test, axis=1)

print("\n--- Sınıflandırma Raporu ---")
print(classification_report(y_true, pred_classes, target_names=labels))

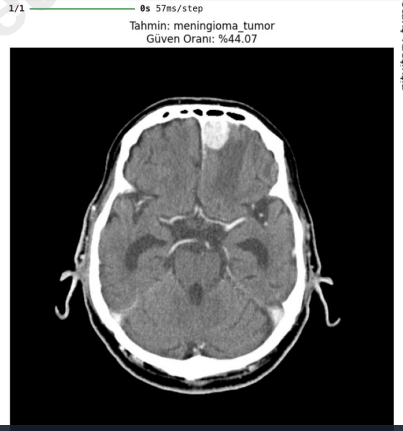
plt.figure(figsize=(10, 8))
sns.heatmap(confusion_matrix(y_true, pred_classes), annot=True, fmt='d', cmap='Greens',
            xticklabels=labels, yticklabels=labels)
plt.title('Beyin Tümörü Karmaşıklık Matrisi')
plt.show()
```

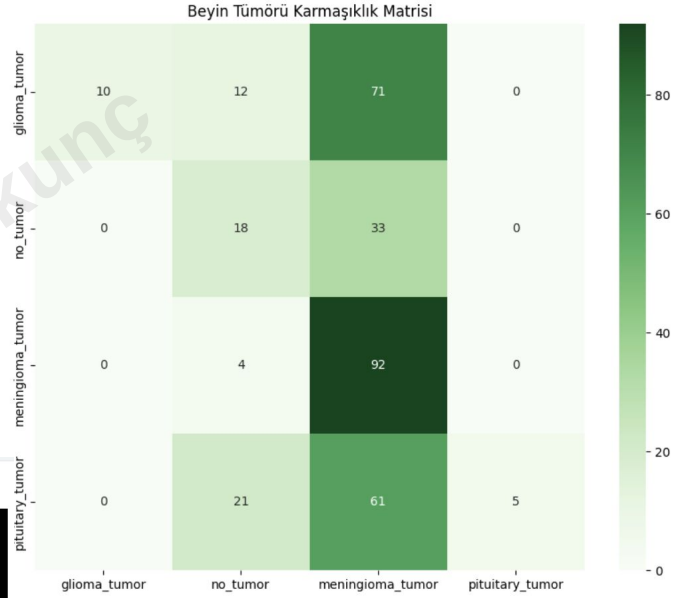
```
11/11 ━━━━━━━━━━━━━━━━━━━━ 18s 940ms/step

--- Sınıflandırma Raporu ---
                  precision    recall  f1-score   support

    glioma_tumor       1.00      0.11      0.19        93
       no_tumor        0.33      0.35      0.34        51
meningioma_tumor       0.36      0.96      0.52        96
 pituitary_tumor       1.00      0.06      0.11        87

        accuracy                           0.38       327
       macro avg       0.67      0.37      0.29       327
    weighted avg       0.71      0.38      0.29       327
```





Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

Projenin Amacı ve Önemi

- Küresel Sorun: İklim değişikliği ve sera gazı emisyonlarının artması.
- Hedef: Birleşmiş Milletler tarafından sunulan "International Greenhouse Gas Emissions" veri seti üzerinde analizler yaparak gelecekteki emisyon sınıflarını tahmin etmek.
- Teknik Yaklaşım: Python kodlama dili kullanılarak geliştirilen, sınıflandırma tabanlı bir yapay zeka algoritması.

Veri Seti Tanımı

- Kaynak: Kaggle - International Greenhouse Gas Emissions.
- Kapsam: 1990 - 2017 yılları arasındaki farklı ülkelerin emisyon verileri.
- Problem Türü: Sınıflandırma (Classification).
- Giriş Parametreleri: Yıllara göre emisyon miktarları ve ülke verileri.
- Çıkış Parametresi: Emisyon Sınıfı.

Muhammed Emin Korkunç

Bu sunumun kopyalanması, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

Yarışma Kuralları ve Metodoloji

- Geliştirme Ortamı: Sadece Python dili ve ilgili kütüphaneler (TensorFlow, Scikit-learn, vb.).
- Veri Ayrımı: Veri setinin %50'si eğitim, %50'si test amaçlı kullanılmıştır.
- Rastgelelik: Eğitim ve test setleri rastgele verilerle oluşturulmuştur.
- Değerlendirme: Farklı çalıştırmalar sonucunda elde edilen ortalama doğruluk değerleri baz alınmıştır.

Yapay Zeka Teknikleri ve Karşılaştırma

- Kullanılan Teknikler: Yarışma gereği en az 2 farklı teknik uygulanmıştır:
  1. Yapay Sinir Ağları (ANN): Karmaşık verilerdeki derin örüntüleri yakalamak için.
  2. Karar Ağaçları / Random Forest: Sayısal verilerde hızlı ve etkili sınıflandırma için.
- Performans Kriterleri: İki tekniğin başarı oranları karşılaştırılarak en iyi sonuç veren model seçilmiştir.

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

Proje Bulguları (Doğruluk)

- Doğruluk (Accuracy): Modelin genel başarı yüzdesi ekrana yansıtılmalıdır.
- Hedeflenen Başarı:
  - Ortalama %70'den yüksek doğruluk (10 Puan).
  - Ortalama %80'den yüksek doğruluk (20 Puan).
  - Ortalama %90'dan yüksek doğruluk (30 Puan).

Karmaşıklık Matrisi (Confusion Matrix)

- Analiz: Modelin hangi emisyon sınıflarını doğru bildiği, hangilerini karıştırdığı görselleştirilmiştir.
- Hata Payı: Yanlış tahminlerin hangi sınıflarda yoğunlaştığının belirlenmesi ve modelin buna göre optimize edilmesi.

Sonuç ve Değerlendirme

- Yapay zeka tekniklerinin emisyon tahminindeki başarısı kanıtlanmıştır.
- Bütün teknikler ile elde edilen sonuçlar karşılaştırılarak en yüksek puanı getiren model raporlanmıştır.
- Bu çalışma, iklim değişikliği ile mücadelede veri odaklı çözümler üretilebileceğini göstermektedir.

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc

Yapay Zekada Etik ve Sorumlu Kullanım

- Neden Önemli?
  - Yapay zeka modellerinin tarafsız (bias-free) ve adil kararlar vermesini sağlamak.
  - Kişisel verilerin gizliliğini korumak ve veri güvenliğini ön planda tutmak.
- Temel Etik İlkeler:
  - Şeffaflık: Algoritmaların nasıl karar verdiğinin açıklanabilir olması.
  - Hesap Verilebilirlik: Yapay zeka çıktılarının sorumluluğunun üstlenilmesi.
  - Doğruluk: Yanıltıcı veya "halüsinasyon" (uydurma) içeriklerin farkında olunması.

Muhammed Emin Korkunç

Bu sunumun kopyalanmasi, paylasilmasi ve yazili izinsiz kullanimi KESINLIKLE YASAKTIR.

Muhammed Emin Korkunç | muhammedemin.korkunc@gmail.com | linkedin.com/in/muhammed-emin-korkunç-100ba2215 | github.com/muhammedkorkunc